

TOY WORLD – E-COMMERCE DATABASE MANAGEMENT SYSTEM

PAYMENT MANAGEMENT SYSTEM

1. Database Design

The Payment entity is designed to store and manage payment transaction details for customer orders in the Toy World – E-commerce Database Management System. It records information such as Payment ID, Order ID, Payment Method, Payment Status, Payment Date, and Amount. The Payment_ID uniquely identifies each payment transaction and acts as the Primary Key. The Order_ID connects the Payment table with the Orders table and acts as a Foreign Key. The Payment entity is related to the Order entity through a One-to-One relationship, with total participation from both entities. The Payment table helps to maintain payment transaction details and manage payment status for orders.

1.1 Payment Table

Attribute	Key Type
Payment_ID	Primary Key
Order_ID	Foreign Key
Payment_Method	-
Payment_Status	-
Payment_Date	-
Amount	-

2. Create Payment Table

```
SQL> CREATE TABLE Payment (  
2 Payment_ID NUMBER PRIMARY KEY,  
3 Order_ID NUMBER,  
4 Payment_Method VARCHAR2(50),  
5 Payment_Status VARCHAR2(30),  
6 Payment_Date DATE,  
7 Amount NUMBER(10,2),  
8 CONSTRAINT fk_payment_order  
9 FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID)  
10 );
```

Table created.

3. Payment Records

```
SQL> INSERT INTO Payment
  2 VALUES (1, 1001, 'UPI', 'Successful', DATE '2026-09-01', 599.00);

1 row created.

SQL>
SQL> INSERT INTO Payment
  2 VALUES (2, 1002, 'Credit Card', 'Successful', DATE '2026-09-02', 899.00);

1 row created.

SQL>
SQL> INSERT INTO Payment
  2 VALUES (3, 1003, 'Debit Card', 'Failed', DATE '2026-09-03', 450.00);

1 row created.

SQL>
SQL> INSERT INTO Payment
  2 VALUES (4, 1004, 'Cash on Delivery', 'Successful', DATE '2026-09-04', 750.00);

1 row created.

SQL>
SQL> INSERT INTO Payment
  2 VALUES (5, 1005, 'UPI', 'Successful', DATE '2026-09-05', 1200.00);

1 row created.

SQL>
SQL> INSERT INTO Payment
  2 VALUES (6, 1006, 'Credit Card', 'Failed', DATE '2026-09-06', 650.00);

1 row created.

SQL>
SQL> INSERT INTO Payment
  2 VALUES (7, 1007, 'Debit Card', 'Successful', DATE '2026-09-07', 999.00);

1 row created.
```

4. Display Payment Records

```
SQL> SELECT
  2   Payment_ID,
  3   Order_ID,
  4   Payment_Method,
  5   Payment_Status,
  6   Payment_Date,
  7   Amount
  8 FROM Payment
  9 ORDER BY Payment_ID;
```

PAYMENT_ID	ORDER_ID	PAYMENT_METHOD	PAYMENT_STATUS	PAYMENT_D	AMOUNT
1	1001	UPI	Successful	01-SEP-26	599
2	1002	Credit Card	Successful	02-SEP-26	899
3	1003	Debit Card	Failed	03-SEP-26	450
4	1004	Cash on Delivery	Successful	04-SEP-26	750
5	1005	UPI	Successful	05-SEP-26	1200
6	1006	Credit Card	Failed	06-SEP-26	650
7	1007	Debit Card	Successful	07-SEP-26	999
8	1008	UPI	Successful	08-SEP-26	550

```
8 rows selected.
```

5. Successful Transactions

```
SQL> SELECT Payment_ID,
2      Order_ID,
3      Payment_Method,
4      Payment_Date,
5      Amount
6 FROM Payment
7 WHERE Payment_Status = 'Successful'
8 ORDER BY Payment_ID;
```

PAYMENT_ID	ORDER_ID	PAYMENT_METHOD	PAYMENT_D	AMOUNT
1	1001	UPI	01-SEP-26	599
2	1002	Credit Card	02-SEP-26	899
4	1004	Cash on Delivery	04-SEP-26	750
5	1005	UPI	05-SEP-26	1200
7	1007	Debit Card	07-SEP-26	999
8	1008	UPI	08-SEP-26	550

6 rows selected.

6. Failed Transactions

```
SQL> SELECT Payment_ID,
2      Order_ID,
3      Payment_Method,
4      Payment_Date,
5      Amount
6 FROM Payment
7 WHERE Payment_Status = 'Failed'
8 ORDER BY Payment_ID;
```

PAYMENT_ID	ORDER_ID	PAYMENT_METHOD	PAYMENT_D	AMOUNT
3	1003	Debit Card	03-SEP-26	450
6	1006	Credit Card	06-SEP-26	650

7. Modify Payment Status

```
SQL> UPDATE Payment
2 SET Payment_Status = 'Successful'
3 WHERE Payment_ID = 5;

1 row updated.

SQL> UPDATE Payment
2 SET Payment_Status = 'Successful'
3 WHERE Payment_ID = 7;

1 row updated.
```

8. Analyze payment methods

8.1 Count Transactions by Payment Method

```
SQL> SELECT Payment_Method,
2         COUNT(Payment_ID) AS Total_Transactions
3 FROM Payment
4 GROUP BY Payment_Method
5 ORDER BY Payment_Method;
```

PAYMENT_METHOD	TOTAL_TRANSACTIONS
Cash on Delivery	1
Credit Card	2
Debit Card	2
UPI	3

8.2 Total Amount by Payment Method

```
SQL> SELECT Payment_Method,
2         SUM(Amount) AS Total_Amount
3 FROM Payment
4 GROUP BY Payment_Method
5 ORDER BY Payment_Method;
```

PAYMENT_METHOD	TOTAL_AMOUNT
Cash on Delivery	750.00
Credit Card	1549.00
Debit Card	1449.00
UPI	2349.00

9. Payment Transaction Report

```
SQL> SELECT p.Payment_ID,
2         p.Order_ID,
3         p.Payment_Method,
4         p.Payment_Status,
5         p.Payment_Date,
6         p.Amount
7 FROM Payment p
8 ORDER BY p.Payment_ID;
```

PAYMENT_ID	ORDER_ID	PAYMENT_METHOD	PAYMENT_STATUS	PAYMENT_D	AMOUNT
1	1001	UPI	Successful	01-SEP-26	599
2	1002	Credit Card	Successful	02-SEP-26	899
3	1003	Debit Card	Failed	03-SEP-26	450
4	1004	Cash on Delivery	Successful	04-SEP-26	750
5	1005	UPI	Successful	05-SEP-26	1200
6	1006	Credit Card	Failed	06-SEP-26	650
7	1007	Debit Card	Successful	07-SEP-26	999
8	1008	UPI	Successful	08-SEP-26	550

8 rows selected.